Domain-Driven Design (DDD) Analysis Report: Timesheet Lite

This report outlines the bounded contexts, core domain entities, value objects, and execution rules for the `timesheet-lite` project based on the current product requirements.

---

1. Bounded Contexts & Classifications

The system is decomposed into distinct Bounded Contexts based on processing load, concurrency needs, and business boundaries:

Bounded Context	Responsibility	Classification
Timesheet Logging Context	Handles weekly calendar loading, hourly adjustments (0-8h sliders), and daily limit validation.	**Lightweight User Interaction**
Verification Context	Handles Team Leader approvals, verifying employee daily totals, and locking approved records.	**Lightweight User Interaction**
Metadata CRUD Context	Administrative management of projects, user accounts, and exceptions calendar.	**Lightweight User Interaction**
Reporting & Export Context	Consolidates user hours, aggregates by week/month, and formats grids into downloadable Excel (`.xlsx`) files.	**Heavy-Duty Processing**
Scheduler & Mail Compliance Context	Automated checking for timesheet submission gaps, approval gaps, and sending SMTP compliance reminders.	**Background & Scheduler**

Bounded Context	Responsibility	Classification
System Operations Context	Database backup generation via SQLite `VACUUM INTO`, backup restoration, WAL cleanup, and Super Code security verification.	**Background & System Ops**

---

2. Core Domain Entities & Attributes

A. Core Entities

- **User**:
  - Unique Identifier (ID): Database primary key (`id`)
  - Key Attributes: `username`, `email`, `full_name`, `cost_center`, `role` (Value Object), `is_deleted` (Soft-delete flag), `team_leader_id` (FK to User).
  - Domain Rule: Standard Employees can only read/edit their own hours; Team Leaders manage employees assigned to them.
- **Project**:
  - Unique Identifier (ID): Database primary key (`id`)
  - Key Attributes: `name`, `status` (Value Object), `custom_id`, `is_default`, `is_deleted`.
  - Domain Rule: Assigned projects (`UserProjectLink`) and default projects are visible to the employee.
- **Timesheet**:
  - Unique Identifier (ID): Database primary key (`id`)
  - Key Attributes: `user_id` (FK to User), `project_id` (FK to Project), `date` (Date), `hours` (Float), `verify` (Boolean).
  - Domain Rule: Deduplicated dynamically (0-hour records are auto-deleted). Once `verify = True`, edits are locked.
- **WorkDay**:
  - Unique Identifier (ID): Date (`date`, primary key)
  - Key Attributes: `day_type` (Value Object), `remark`.
  - Domain Rule: Overrides default calendar rules (Mon-Fri = 8h capacity, Sat-Sun = 0h capacity).
- **ActivityLog**:
  - Unique Identifier (ID): Database primary key (`id`)
  - Key Attributes: `user_id` (FK to User), `action` (String), `details`, `timestamp`.

B. Value Objects (Immutable Status & Roles)

- **Role**: `admin`, `team_leader`, `employee`

- **WorkDayType**:
  - `work` (8 hours capacity)
  - `half_off` (4 hours capacity)
  - `off` (0 hours capacity)
- **ProjectStatus**: `RUN`, `CLOSE`, `NOT START`

---

3. Business Invariants & Work Hour Constraints

1. **Daily Hour Dynamic Constraint**:
   - The total hours logged by a `user_id` on a specific `date` must not exceed the capacity dictated by the exception calendar:

     Daily Hours logged ≤ WorkDayType capacity (WORK=8h, HALF_OFF=4h, OFF=0h)
2. **Weekly Hour Dynamic Constraint**:
   - The sum of hours logged for a week (Monday to Sunday) must not exceed the dynamic weekly limit:

     $$\text{Weekly Total Hours} \leq \sum_{d \in \text{week}} \text{Capacity}(d)$$
3. **Write Modification Cutoffs (Edit Windows)**:
   - For standard Employees, timesheet logs older than **2 weeks** (for single entry edits) or **30 days** (for batch edits) cannot be modified or written.
   - Verified records (`verify = True`) cannot be modified by standard Employees.

---

4. Execution & Offloading Strategy

To optimize SQLite performance (WAL mode) and ensure frontend UI responsiveness, operations are delegated as follows:

- **Asynchronous / Non-blocking Routing**:
  - Backups (`/api/backup/`) and restorations (`/api/restore/`) must execute inside background threads using FastAPI `BackgroundTasks`.
  - Restoring a backup requires disposing of the engine to close socket connections, replacing the file, and deleting old `-wal` and `-shm` transaction files to avoid data corruption.
- **Background Cron Jobs**:
  - Compliance routines (`run_timesheet_check`, `run_approval_check`) run on Monday at 10:00 AM using `APScheduler`. Compliance messages are dispatched asynchronously to prevent blocking.
- **Frontend Optimization**:
  - Reports table rendering must use pagination in Element Plus to prevent rendering blockages when loading large datasets.